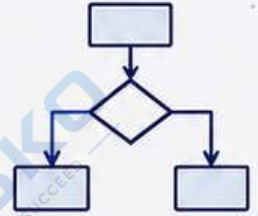


1010101010
0101010101
1010101010
0101010101



10110
01001
10101

Question Bank



Advance Python Programming

Unit I - Strings, Lists and Dictionaries

- **Answers in one or two lines (1 mark questions)**

1. What is a string in Python?
2. What is string indexing?
3. What is string slicing?
4. What is string formatting?
5. Are strings mutable or immutable in Python?
6. What is a list in Python?
7. How do you create a list in Python?
8. What is list indexing?
9. What is list slicing?
10. Name any two built-in functions used with lists.
11. Name any two list methods.
12. What is a dictionary in Python?
13. What is a key-value pair?
14. How do you create a dictionary in Python?
15. How can you access a value from a dictionary?
16. How can you modify a dictionary value?
17. Name any two dictionary methods.
18. What is the use of the `del` statement?
19. Are dictionaries mutable in Python?
20. Are dictionary keys required to be unique?

- **Short Answer Questions (3 marks questions)**

1. Explain basic string operations in Python with examples.
2. Explain string indexing with suitable examples.
3. Explain string slicing with suitable examples.
4. Explain string formatting in Python with examples.
5. Explain any five commonly used string methods.
6. Explain how lists are created in Python.
7. Explain list indexing and slicing with examples.
8. Explain different operations that can be performed on lists.
9. Explain any five built-in functions used with lists.
10. Explain any five list methods with examples.
11. Explain dictionaries and their structure with an example.
12. Explain how to create, access, and modify dictionary key-value pairs.
13. Explain built-in functions used with dictionaries.
14. Explain commonly used dictionary methods.
15. Explain the `del` statement with suitable examples.

- **Long Answer Questions (6 marks questions)**

1. Explain strings in Python and discuss string operations, indexing, slicing, formatting, and commonly used string methods with suitable examples.
2. Explain string indexing and slicing in detail with suitable examples.
3. Explain string formatting techniques in Python with suitable examples.
4. Explain lists in Python, including creation, operations, indexing, slicing, built-in functions, and list methods.
5. Explain different list operations and list methods with suitable examples.
6. Explain dictionaries in Python, including creation, accessing, modifying, and deleting key-value pairs.
7. Explain built-in functions and commonly used methods of dictionaries with suitable examples.
8. Differentiate between Strings, Lists, and Dictionaries with suitable examples.
9. Write a detailed note on indexing and slicing in both strings and lists with examples.

Unit II - Tuples and Sets

- **Answers in one or two lines (1 mark questions)**

1. What is a tuple in Python?
2. How do you create a tuple?
3. Are tuples mutable or immutable?
4. What is tuple indexing?
5. What is tuple slicing?
6. Name any two built-in functions used with tuples.
7. Name the two methods available for tuples.
8. What is the relation between tuples and lists?
9. What is a set in Python?
10. Does a set allow duplicate elements?
11. How do you create an empty set?
12. Name any two set methods.
13. What is set traversal?
14. What is a frozen set?
15. What is the `zip()` function?
16. What does the `len()` function return for a tuple?
17. Can a tuple contain different data types?
18. Can a set contain duplicate elements?
19. Can elements of a set be accessed using an index?
20. What is the difference between a set and a frozen set?

- **Short Answer Questions (3 marks questions)**

1. Explain tuples in Python with an example.
2. Explain how to create tuples.
3. Explain basic tuple operations with examples.
4. Explain tuple indexing with suitable examples.
5. Explain tuple slicing with suitable examples.
6. Explain built-in functions used with tuples.
7. Explain the relationship between tuples and lists.
8. Explain the relationship between tuples and dictionaries.
9. Explain tuple methods with examples.
10. Explain the `zip()` function with a suitable example.
11. Explain sets in Python with an example.
12. Explain commonly used set methods.
13. Explain set traversal with an example.
14. Explain frozen sets with an example.
15. Differentiate between tuples and sets.

- **Long Answer Questions (6 marks questions)**

1. Explain tuples in Python, including creation, operations, indexing, slicing, built-in functions, and methods with examples.
2. Explain basic tuple operations, indexing, and slicing with suitable examples.
3. Explain the relationship between tuples and lists with suitable examples.
4. Explain the relationship between tuples and dictionaries with suitable examples.
5. Explain the `zip()` function in detail with suitable examples.
6. Explain sets in Python, including creation, operations, methods, and traversal.
7. Explain different set methods with suitable examples.
8. Explain frozen sets, their characteristics, and applications.
9. Differentiate between tuples, lists, sets, and frozen sets.
10. Write a Python program demonstrating tuple operations, `zip()`, set operations, and set traversal.

Unit III - Files and Regular Expression Operations

- **Answers in one or two lines (1 mark questions)**

1. What is a file in Python?
2. Name the two main types of files.
3. What is a text file?
4. What is a binary file?
5. Which function is used to open a file in Python?
6. What is the purpose of the `open()` function?
7. What is the `read()` method?
8. What is the `write()` method?
9. What is the `close()` method?
10. What is the purpose of the `with` statement when working with files?
11. What is the Pickle module?
12. What is serialization?
13. What is deserialization?
14. What is a CSV file?
15. Which Python module is commonly used to work with CSV files?
16. What is the `os` module?
17. What is the `os.path` module?
18. What is a Regular Expression?
19. What is the purpose of the `re` module?
20. What is the `glob` module?

- **Short Answer Questions (3 marks questions)**

1. Explain the different types of files in Python.
2. Explain how to create and open a text file.
3. Explain `read()`, `readline()`, and `readlines()` methods.
4. Explain different file modes used with `open()`.
5. Explain `write()` and `writelines()` methods with examples.
6. Explain reading and writing binary files.
7. Explain the Pickle module in Python.
8. Explain serialization and deserialization with examples.
9. Explain reading and writing CSV files.
10. Explain the `os` module with suitable examples.
11. Explain the `os.path` module and its commonly used functions.
12. Explain Regular Expressions and their applications.
13. Explain special characters used in Regular Expressions.
14. Explain commonly used Regular Expression methods.
15. Explain named groups in Python Regular Expressions.
16. Explain the use of Regular Expressions with the `glob` module.

- **Long Answer Questions (6 marks questions)**

1. Explain file handling in Python with different file types, file modes, and file operations.
2. Explain creating, reading, writing, and closing text files with suitable Python programs.
3. Explain different file methods such as `read()`, `readline()`, `readlines()`, `write()`, and `writelines()` with examples.
4. Explain binary file handling and the Pickle module with suitable examples.
5. Explain serialization and deserialization using the Pickle module.
6. Explain reading and writing CSV files in Python with suitable programs.
7. Explain the `os` and `os.path` modules and their commonly used functions.
8. Explain Regular Expressions, special characters, and Regular Expression methods with suitable examples.

9. Explain named groups in Python Regular Expressions with suitable examples.
10. Explain the use of Regular Expressions with the `glob` module with suitable examples.

Unit IV - Object-Oriented Programming in Python

● Answers in one or two lines (1 mark questions)

1. What is Object-Oriented Programming?
2. What is a class in Python?
3. What is an object?
4. How do you create a class in Python?
5. How do you create an object in Python?
6. What is the `__init__()` method?
7. What is a constructor?
8. What is a class attribute?
9. What is a data attribute?
10. What is the difference between class attributes and data attributes?
11. What is encapsulation?
12. What is inheritance?
13. What is polymorphism?
14. What is a parent class?
15. What is a child class?
16. What is method overriding?
17. Which keyword is used to define a class in Python?
18. What is the purpose of the `self` parameter?
19. Can a class have multiple objects?
20. What is the advantage of Object-Oriented Programming?

● **Short Answer Questions (3 marks questions)**

1. Explain Object-Oriented Programming in Python.
2. Explain classes and objects with a suitable example.
3. Explain how to create a class in Python.
4. Explain how to create objects in Python.
5. Explain the constructor method `__init__()` with an example.
6. Explain the `self` parameter in Python.
7. Explain how multiple objects can be created from a class.
8. Explain class attributes with a suitable example.
9. Explain data attributes with a suitable example.
10. Differentiate between class attributes and data attributes.
11. Explain encapsulation in Python.
12. Explain inheritance in Python with an example.
13. Explain polymorphism in Python with an example.
14. Explain method overriding in Python.
15. Explain the advantages of Object-Oriented Programming in Python.

● **Long Answer Questions (6 marks questions)**

1. Explain Object-Oriented Programming in Python and its major concepts in detail.
2. Explain classes and objects with suitable Python programs.
3. Explain the creation of classes and objects with the constructor method `__init__()`.
4. Explain the constructor method and `self` parameter with suitable examples.
5. Explain classes with multiple objects using a suitable Python program.
6. Explain class attributes and data attributes with suitable examples.
7. Differentiate between class attributes and data attributes with a suitable program.
8. Explain encapsulation in Python with suitable examples.
9. Explain inheritance in Python with suitable examples.
10. Explain polymorphism in Python with suitable examples.